

6 Killer function

Javascript



@skrchowdhury

1. Debounce Function

A debounce function limits the rate at which a function can fire. This is especially useful for optimizing performance in events like window resizing, scrolling, or keystroke events.



```
1  function debounce(func, delay) {
2    let timeout;
3    return function(...args) {
4      clearTimeout(timeout);
5      timeout = setTimeout(() => func.apply(this, args), delay);
6    };
7  }
8
9  // Usage
10 const log = debounce(() => console.log('Debounced!'), 2000);
11 window.addEventListener('resize', log);
12
```

2. Throttle Function

A throttle function ensures that a function is only called at most once in a specified period. This is useful for events that can fire rapidly, such as scrolling or resizing.

```
1  function throttle(func, limit) {
2    let lastFunc;
3    let lastRan;
4    return function(...args) {
5      if (!lastRan) {
6        func.apply(this, args);
7        lastRan = Date.now();
8      } else {
9        clearTimeout(lastFunc);
10       lastFunc = setTimeout(() => {
11         if ((Date.now() - lastRan) >= limit) {
12           func.apply(this, args);
13           lastRan = Date.now();
14         }
15       }, limit - (Date.now() - lastRan));
16     }
17   };
18 }
19
20 // Usage
21 const log = throttle(() => console.log('Throttled!'), 2000);
22 window.addEventListener('scroll', log);
23
```

3. Currying Function

Currying is a functional programming technique that transforms a function with multiple arguments into a sequence of functions, each taking a single argument.

```
1  function curry(func) {
2    return function curried(...args) {
3      if (args.length >= func.length) {
4        return func.apply(this, args);
5      }
6      return function(...args2) {
7        return curried.apply(this, [...args, ...args2]);
8      };
9    };
10 }
11
12 // Usage
13 function sum(a, b, c) {
14   return a + b + c;
15 }
16 const curriedSum = curry(sum);
17 console.log(curriedSum(1)(2)(3)); // Output: 6
18
```

4. Memoization Function

Memoization is an optimization technique that stores the results of expensive function calls and returns the cached result when the same inputs occur again.

```
1  function memoize(func) {
2    const cache = {};
3    return function(...args) {
4      const key = JSON.stringify(args);
5      if (cache[key]) {
6        return cache[key];
7      }
8      const result = func.apply(this, args);
9      cache[key] = result;
10     return result;
11   };
12 }
13
14 // Usage
15 const fibonacci = memoize(n => (n <= 1 ? n : fibonacci(n - 1) + fibonacci(n - 2)));
16 console.log(fibonacci(40)); // Output: 102334155
17
```

5. Deep Clone Function

A deep clone function creates a new object that is a deep copy of the original object, ensuring that nested objects are also copied.

```
1  function deepClone(obj) {  
2      return JSON.parse(JSON.stringify(obj));  
3  }  
4  
5  // Usage  
6  const original = { a: 1, b: { c: 2 } };  
7  const cloned = deepClone(original);  
8  cloned.b.c = 3;  
9  console.log(original.b.c); // Output: 2  
10
```